Patient Dashboard Application

A full-stack Patient Registration Dashboard built for managing patient registration records, viewing patient information, updating patient status, and displaying real-time dashboard statistics.

This project was developed as part of a full-stack development internship assignment given by Touchstone Institute. The focus of the application is clean architecture, strong TypeScript usage, REST API design, responsive UI, form validation, error handling, and a professional user experience.

Repository

GitHub Repository Link: (https://github.com/Drishtiii18/patient-registration.git)

Table of Contents

- Patient Dashboard Application
  - Repository
    - Table of Contents
    - Project Overview
    - Features
      - Dashboard
      - Add Patient Form
      - Patient Detail Page
    - Tech Stack
      - Frontend
      - Backend
    - Project Structure (Given)
    - Environment Variables
    - Quick Start
      - Backend (quick)
        - Frontend (quick)
    - Backend Setup
    - Frontend Setup
    - Running the Full Application
      - Step 1: Start MongoDB

    - Step 2: Start the Backend
    - Step 3: Start the Frontend
    - Step 4: Open the App
  - Seeding the Database
  - API Documentation
    - Get All Patients
      - Supported Query Parameters
    - Get Patient by ID
    - Create Patient
    - Update Patient Status
    - Get Dashboard Statistics
  - API Status Codes
  - Frontend Pages
  - Form Validation
  - Error Handling
    - Backend Error Handling
    - Frontend Error Handling
  - UI and UX Decisions
  - TypeScript and Code Quality
  - Angular Architecture
  - Backend Architecture
    - Backend Checklist
    - Frontend Checklist
  - Common Issues and Fixes
    - Backend does not start
    - MongoDB connection fails
    - Frontend cannot reach backend
    - Empty dashboard
  - Future Improvements
  - Author

Project Overview

The Patient Dashboard Application allows staff members to manage patient registration records through a clean and responsive web interface.

The application includes:

- A dashboard with patient statistics
- A pie chart showing patient status distribution
- A paginated patient records table
- Search and status filtering
- A form to add new patients
- A patient detail page
- Functionality to update patient status
- Loading states and user-friendly error messages

The project is divided into two main folders:

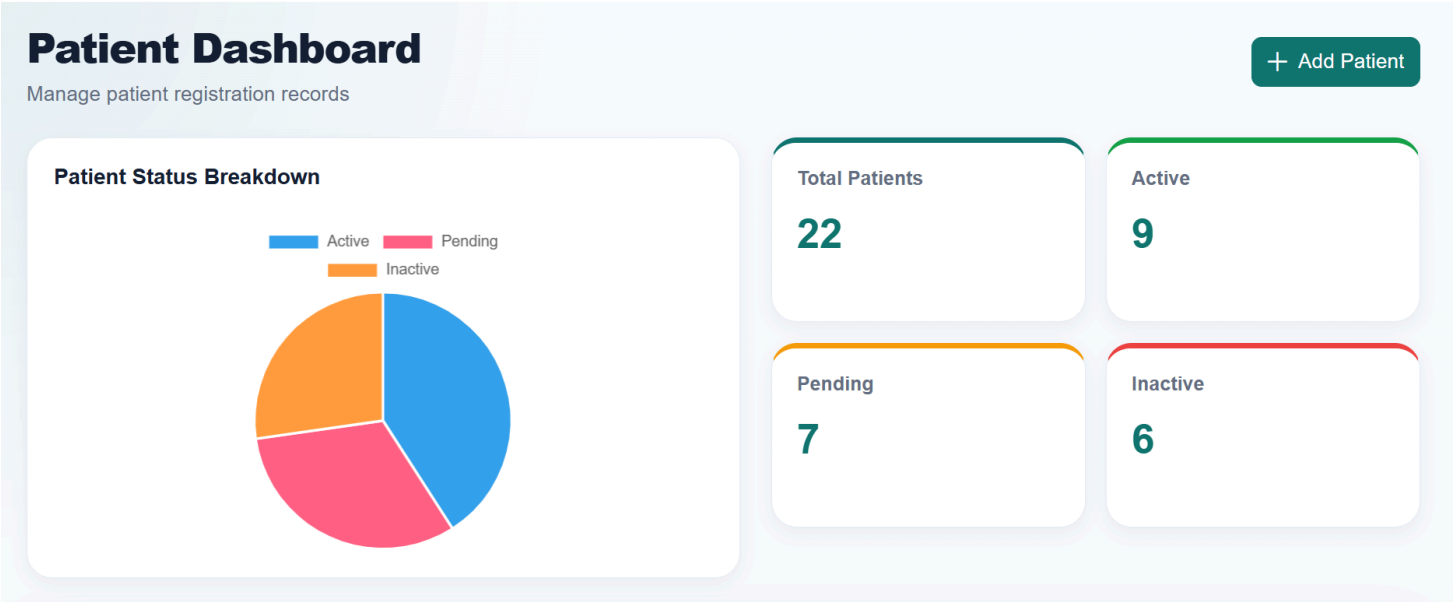
```
backend/
frontend/
```

The backend provides a REST API using Node.js, Express, TypeScript, MongoDB, and Mongoose.

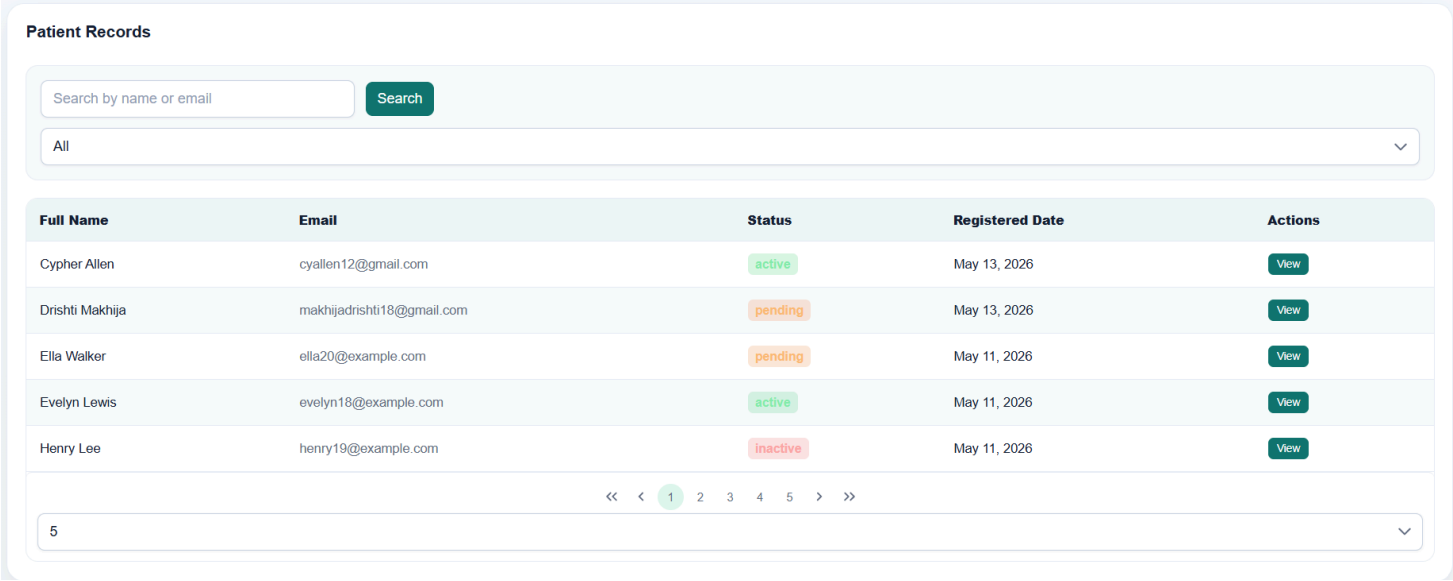
The frontend is built with Angular, TypeScript, PrimeNG, Angular Router, Reactive Forms, and Angular Signals.

Features

Dashboard



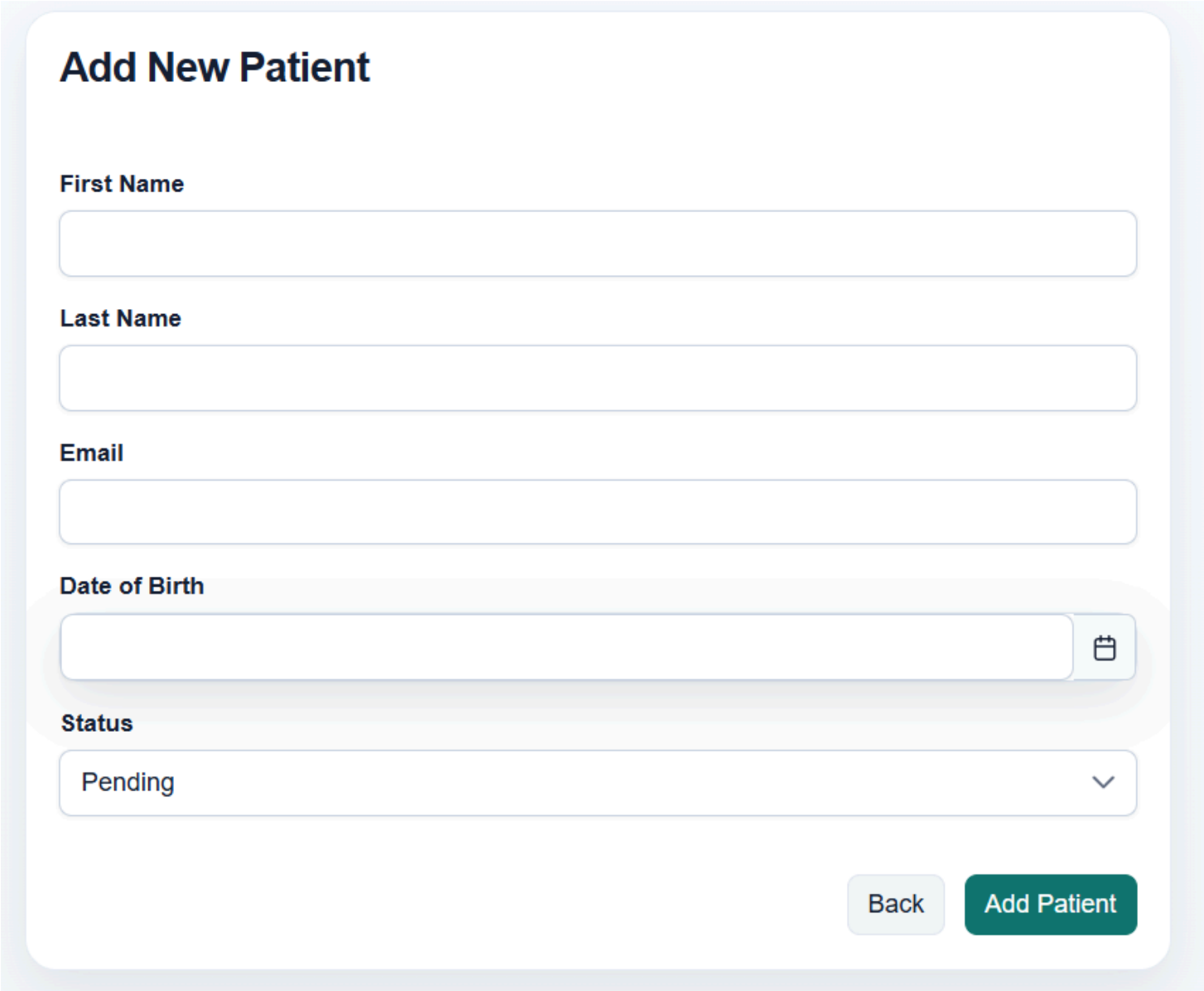
- Displays patient statistics using summary cards
- Shows patient status breakdown using a pie chart



- Displays patient records in a responsive table
- Supports server-side search by name or email
- Supports filtering by patient status
- Supports pagination with multiple page size options
- Provides a button to navigate to the Add Patient form

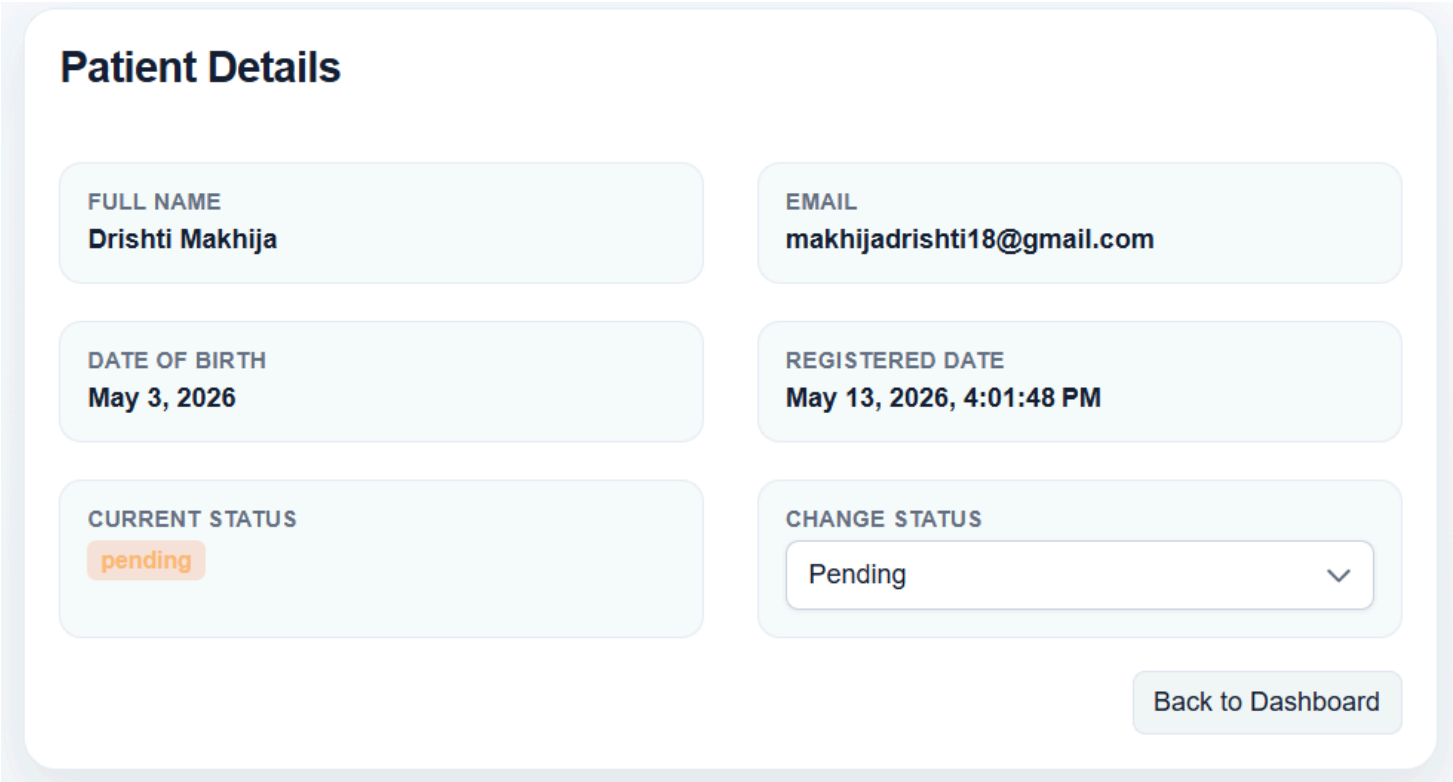
- Displays loading and error states

Add Patient Form



- Allows staff to create a new patient record
- Uses Reactive Forms
- Includes required field validation
- Validates email format
- Prevents future dates of birth
- Keeps form data available if an API error occurs
- Shows loading state while submitting

Patient Detail Page



- Displays full patient information in a read-only layout
- Allows status updates using a dropdown
- Sends status updates to the backend API
- Shows success and error messages
- Includes navigation back to the dashboard

Tech Stack

Frontend

Technology	Purpose
Angular	Frontend framework
TypeScript	Strong typing and maintainable code
PrimeNG	UI components
PrimeIcons	Icons

Technology	Purpose
Angular Router	Client-side routing
Reactive Forms	Form handling and validation
Angular Signals	Local state management
HttpClient	API communication

Backend

Technology	Purpose
Node.js	JavaScript runtime
Express.js	REST API framework
TypeScript	Strong typing
MongoDB	Database
Mongoose	MongoDB ODM
dotenv	Environment variable management
CORS	Frontend-backend communication

Project Structure (Given)

```
patient-dashboard/
├── backend/
│   ├── src/
│   │   ├── data/
│   │   │   └── seed.ts
│   │   ├── models/
│   │   │   └── patient.model.ts
│   │   ├── routes/
│   │   │   ├── patients.routes.ts
│   │   │   └── stats.routes.ts
│   │   └── index.ts
│   ├── .env
│   ├── package.json
│   └── tsconfig.json
│
├── frontend/
│   ├── src/
│   │   └── app/
│   │       ├── models/
│   │       │   └── patient.model.ts
│   │       ├── pages/
│   │       │   ├── dashboard/
│   │       │   ├── patient-form/
│   │       │   └── patient-detail/
│   │       ├── shared/
│   │       │   ├── components/
│   │       │   │   ├── spinner/
│   │       │   │   └── status-badge/
│   │       │   └── services/
│   │       │       └── patient.service.ts
│   │       └── app.routes.ts
│   ├── package.json
│   └── tsconfig.json
│
│
└──images
│
└── README.md
```

Environment Variables

Create a `.env` file inside the `backend/` folder. (to setup the environment)

```
MONGODB_URI=your_mongodb_connection_string
PORT=5000
```

Example:

```
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/patient-dashboard
PORT=5000
```

Key points:

- The `.env` file should not be committed to GitHub.
- The MongoDB connection string should never be hardcoded in the source code.
- Make sure `.env` is included in `.gitignore`.

Recommended `.gitignore` entry:

```
.env
node_modules
dist
```

Quick Start

Backend (quick)

```
cd backend
npm install
npm run dev
```

Frontend (quick)

```
cd frontend
npm install
ng serve
```

Backend Setup

Open a terminal and navigate to the backend folder:

```
cd backend
```

Install backend dependencies:

```
npm install
```

Start the backend development server:

```
npm run dev
```

The backend should run at:

```
http://localhost:5000
```

To build the backend:

```
npm run build
```

To run the compiled backend:

```
npm start
```

Frontend Setup

Open a new terminal and navigate to the frontend folder:

```
cd frontend
```

Install frontend dependencies:

```
npm install
```

Start the Angular development server:

```
ng serve
```

The frontend should run at:

```
http://localhost:4200
```

Running the Full Application

To run the complete application locally:

Step 1: Start MongoDB

Use either:

- MongoDB Atlas
- Local MongoDB Community Server

Make sure the connection string is added to:

```
backend/.env
```

Step 2: Start the Backend

```
cd backend
npm install
npm run dev
```

Backend URL:

```
http://localhost:5000
```

Step 3: Start the Frontend

Open a second terminal:

```
cd frontend
npm install
ng serve
```

Frontend URL:

```
http://localhost:4200
```

Step 4: Open the App

Open the following URL in your browser:

```
http://localhost:4200
```

Seeding the Database

To populate the database with sample patients, run:

```
cd backend
npm run seed
```

The seed script inserts sample patient records with different statuses:

- active
- pending
- inactive

This makes the dashboard, chart, filtering, pagination and statistics easier to test.

API Documentation

Base backend URL:

```
http://localhost:5000
```

Get All Patients

```
GET /api/patients
```

Returns a paginated list of patients.

Supported Query Parameters

Parameter	Description	Example
search	Search by name or email	`search=john`
status	Filter by status	`status=active`
page	Page number	`page=1`
pageSize	Number of records per page	`pageSize=10`

Example request:

```
GET /api/patients?search=john&status=active&page=1&pageSize=10
```

Example response:

```
{
  "data": [
    {
      "_id": "patient_id",
      "firstName": "John",
      "lastName": "Doe",
      "email": "john@example.com",
      "dateOfBirth": "1998-05-10",
      "status": "active",
      "registeredDate": "2026-05-11T14:20:00.000Z"
    }
  ],
  "total": 1,
  "page": 1,
  "pageSize": 10
}
```

Get Patient by ID

```
GET /api/patients/:id
```

Example response:

```
{
  "_id": "patient_id",
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "dateOfBirth": "1998-05-10",
  "status": "active",
  "registeredDate": "2026-05-11T14:20:00.000Z"
}
```

If the patient does not exist:

```
{
  "error": "Patient not found"
}
```

Create Patient

```
POST /api/patients
```

Example request body:

```
{
  "firstName": "Ava",
  "lastName": "Martin",
  "email": "ava.martin@example.com",
  "dateOfBirth": "1999-03-18",
  "status": "pending"
}
```

Example response:

```
{
  "_id": "patient_id",
  "firstName": "Ava",
  "lastName": "Martin",
  "email": "ava.martin@example.com",
  "dateOfBirth": "1999-03-18",
  "status": "pending",
  "registeredDate": "2026-05-11T14:20:00.000Z"
}
```

Update Patient Status

```
PUT /api/patients/:id
```

Example request body:

```
{
  "status": "active"
}
```

Example response:

```
{
  "_id": "patient_id",
  "firstName": "Ava",
  "lastName": "Martin",
  "email": "ava.martin@example.com",
  "dateOfBirth": "1999-03-18",
  "status": "active",
  "registeredDate": "2026-05-11T14:20:00.000Z"
}
```

Get Dashboard Statistics

```
GET /api/stats
```

Example response:

```
{
  "total": 20,
  "active": 8,
  "pending": 7,
  "inactive": 5
}
```

API Status Codes

Status Code	Meaning
200	Request successful
201	Patient created successfully
400	Invalid request data
404	Patient not found

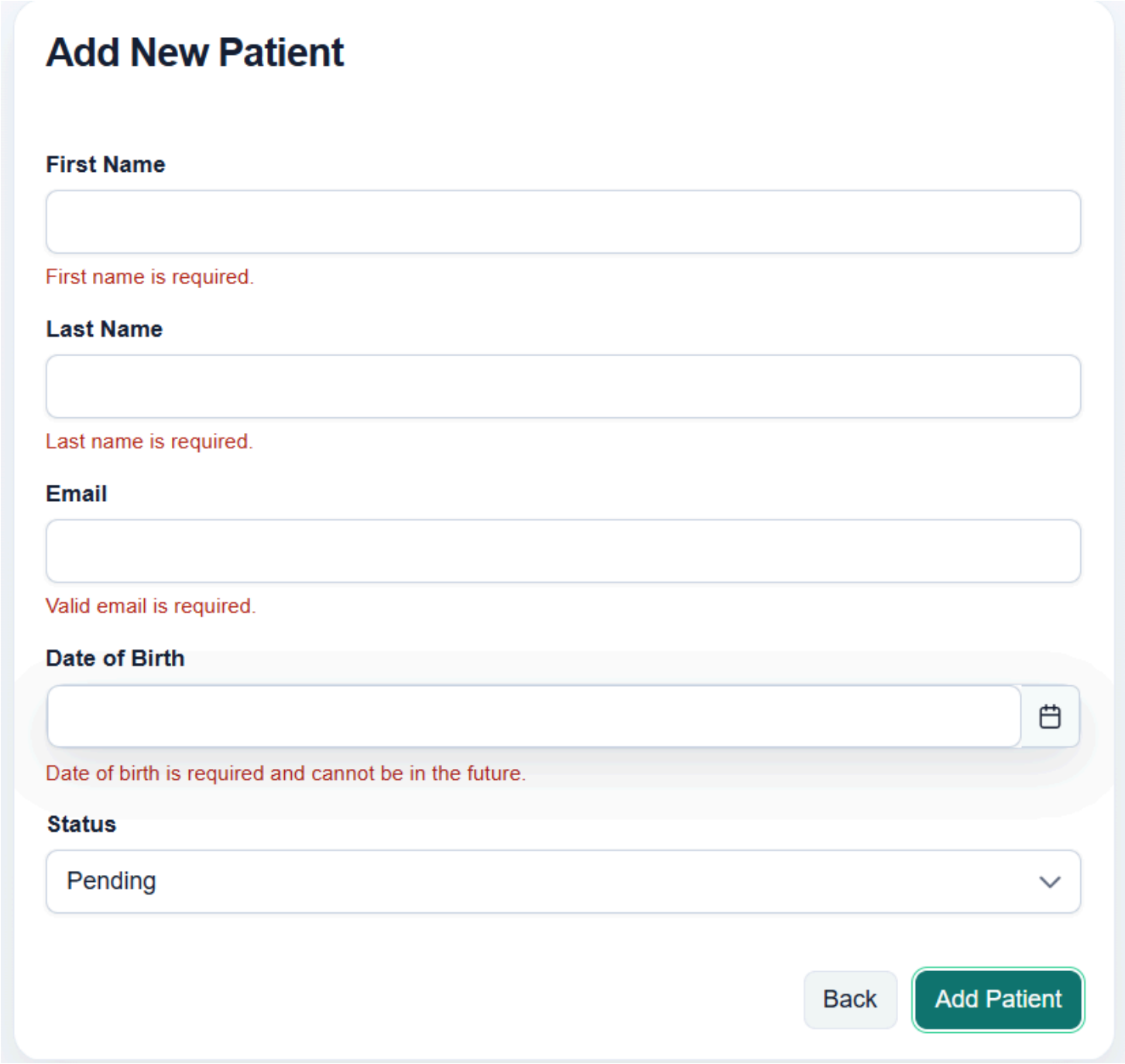
Status Code	Meaning
500	Server error

Frontend Pages

Route	Page	Description
`/dashboard`	Dashboard	Displays stats, chart, filters, and patient table
`/patients/new`	Add Patient	Form for creating a new patient
`/patients/:id`	Patient Details	Displays patient details and allows status update

Form Validation

The Add Patient form includes the following validation rules:



Field	Validation
First Name	Required
Last Name	Required
Email	Required and must be valid
Date of Birth	Required and cannot be in the future
Status	Required

Validation messages are displayed inline to help the user correct mistakes quickly.

Error Handling

The application handles errors in both the frontend and backend.

Backend Error Handling

The backend returns meaningful JSON error messages.

Example:

```
{
  "error": "Patient not found"
}
```

Frontend Error Handling

The frontend displays clear messages when:

- Patients fail to load
- A patient detail record cannot be fetched
- A new patient cannot be created
- A patient status update fails
- Form validation fails

The form remains filled if an API request fails, preventing the user from losing entered data.

UI and UX Decisions

The interface was designed to look professional, clean, and appropriate for a healthcare/admin dashboard.

Key UI decisions:

- Light theme for readability
- White cards with soft shadows
- Teal primary color for a healthcare-inspired look
- Color-coded patient status badges

- Responsive layout for different screen sizes
- Clear form labels and validation messages
- Dedicated loading states
- Dashboard statistics displayed in summary cards
- Pie chart used for quick visual status comparison
- Patient records organized in a paginated table

Status badge colors:

Status	Color Meaning
Active	Green for currently active records
Pending	Amber for records requiring attention
Inactive	Red for inactive records

TypeScript and Code Quality

This project emphasizes clean and maintainable TypeScript code.

Code quality practices include:

- Typed patient model interfaces
- Typed API responses
- Typed service methods
- No direct API calls inside Angular components
- API logic separated into a shared service
- Backend routes organized by feature
- Mongoose schema used for database structure
- Environment variables used for configuration
- Reusable UI components such as status badges and spinner
- Clear naming for files, variables, and functions

Angular Architecture

The frontend follows a clean Angular structure:

- Standalone components
- Lazy-loaded routes
- Angular Signals for local state
- Reactive Forms for form handling
- Shared service for HTTP requests
- Reusable components for spinner and status badge
- PrimeNG components for consistent UI

The goal was to keep components focused on UI behavior while placing API communication inside the patient service.

Backend Architecture

The backend follows a simple REST API architecture:

- `index.ts` initializes the Express server and MongoDB connection
- `patients.routes.ts` handles patient-related API routes
- `stats.routes.ts` handles dashboard statistics
- `patient.model.ts` defines the Mongoose schema and TypeScript interfaces
- `seed.ts` inserts sample records into MongoDB

This separation keeps the backend easy to understand, test, and extend.

Backend Checklist

- [ ] Backend starts without errors
- [ ] MongoDB connects successfully
- [ ] `.env` file is configured correctly
- [ ] Seed script runs successfully
- [ ] `GET /api/patients` returns patient records
- [ ] `GET /api/stats` returns correct counts
- [ ] `GET /api/patients/:id` returns a single patient

- [ ] `POST /api/patients` creates a patient
- [ ] `PUT /api/patients/:id` updates patient status
- [ ] Invalid patient ID returns a 404 response
- [ ] Invalid request data returns a 400 response

Frontend Checklist

- [ ] Dashboard loads successfully
- [ ] Summary cards display correct counts
- [ ] Pie chart displays status breakdown
- [ ] Patient table displays records
- [ ] Search works by name
- [ ] Search works by email
- [ ] Status filter works
- [ ] Pagination works
- [ ] Add Patient button navigates to the form
- [ ] Form validation messages appear
- [ ] New patient can be submitted successfully
- [ ] Patient detail page loads correctly
- [ ] Patient status can be updated
- [ ] Back buttons navigate correctly
- [ ] Loading states appear during API calls
- [ ] Error messages appear when API calls fail

Common Issues and Fixes

Backend does not start

Check that dependencies are installed:

```
npm install
```

Check that the `.env` file exists inside the `backend/` folder.

Check that `MONGODB_URI` is correct.

MongoDB connection fails

Make sure:

- The MongoDB Atlas username and password are correct
- The database user has permission to read and write
- The IP address is allowed in MongoDB Atlas Network Access
- The connection string is copied correctly

Frontend cannot reach backend

Make sure the backend is running at:

```
http://localhost:5000
```

Also check that the frontend service is calling the correct API base URL.

Example:

```
private readonly apiUrl = 'http://localhost:5000/api/patients';
```

Empty dashboard

Run the seed script:

```
cd backend
npm run seed
```

Then refresh the frontend.

Future Improvements

Potential improvements for future versions:

- User authentication and role-based access
- Edit patient profile information
- Delete or archive patient records
- Advanced sorting in the patient table
- More detailed analytics charts
- Export patient records as CSV
- Unit tests for backend routes
- Angular component tests
- Toast notifications for all success and error states
- Deployment to a cloud platform
- API documentation with Swagger/OpenAPI

Author

Drishti Makhija

Full-stack Patient Dashboard Application
Built using Angular, Node.js, Express, TypeScript, MongoDB, Mongoose, and PrimeNG.